Diversified Skyline Explanations for Graph Neural Networks

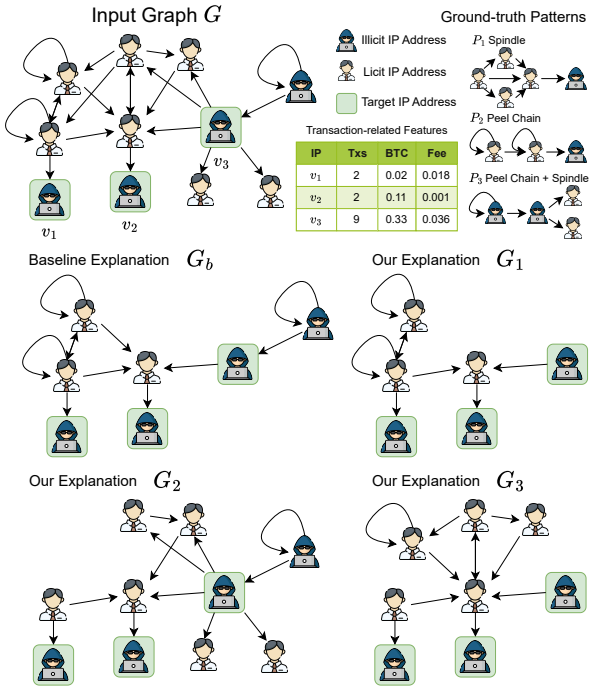
ABSTRACT

We investigate a novel class of skyline explanatory queries to explain and understand the behavior of graph neural networks (GNNs). Existing GNN explainability methods identify subgraphs that are responsible for the GNN's outcome under different explainability measures, e.g., fidelity, conciseness, etc. However, no single explainability measure is the best. To this end, our skyline explanatory query aims to identify k subgraphs that can clarify the inferred results of the GNN from multiple quality measures of explanations simultaneously. (1) We define quality measures for the answers of a GNN's skyline explanatory query, and pursue a multi-objective optimization scheme to compute k subgraphs as answers that approximately achieve a Pareto optimal set. (2) We show the hardness for evaluating skyline explanatory queries. (3) We develop efficient algorithms to compute answers for skyline explanatory queries. The algorithms adopt an edge stream engine to dynamically prioritize the edges in large graphs, and incrementally generate answers that simultaneously introduce the best relative improvement over all explainability measures. (4) We further develop an algorithm to provide diversified answers to skyline explanatory queries, such that the result of GNN inference can be clarified from different perspectives. Using real-world graphs, we experimentally verify the effectiveness, efficiency, and scalability of our algorithms.

1 INTRODUCTION

Graph neural networks (GNNs) have demonstrated promising performances in graph analytical tasks such as classification. Given a graph G, a GNN $\mathcal{M}$ aims to learn the node representations of G that can be converted to proper results for targeted downstream tasks, e.g., node classification, link prediction, or regression analysis. GNNs have been applied for applications such as social networks and biochemistry [11, 49, 50, 55]. Nevertheless, it remains a desirable yet nontrivial task to help users understand, explain, and clarify the output of GNNs [56]. Given $\mathcal{M}$ and a test graph G, a "GNN explainer" generates a critical fraction of G that is "responsible" to clarify the output of $\mathcal{M}$ over G.

Several GNN explainers have been proposed to generate subgraphs to explain the output of GNNs [30, 31, 45, 54, 56, 60]. While these methods serve as general post-process explainers for GNN inference, they are "model-focused", advocating to explain model behavior over a single pre-defined explainability measure. Such methods often fall short at providing more "user-focused" explanations. For example, one may require specific explanations for the output of designated nodes of interests, rather than entire learned representations [36]; another need is to customize the explanation generation for a more comprehensive list of explainability quality measures in terms of fidelity, conciseness, factual, or counterfactual measures [29, 60]. In addition, more comprehensive explanations that clarify GNN output from different perspectives are often more desired to prevent biased learning-based decision making [33]. Existing "model-driven" explainers often are biased and fall short at providing configurable, comprehensive explanation for GNNs.



IP	Txs	BTC	Fee
v_1	2	0.02	0.018
v_2	2	0.11	0.001
v_3	9	0.33	0.036

Figure 1: A bitcoin transaction network with three illicit IP addresses: v_1, v_2, and v_3 (green box) [15]. The ground-truth illicit transaction patterns (e.g. money laundering) are shown as P_1, P_2, and P_3. G_b is an explanation subgraph generated by [54]. G_1, G_2, and G_3 are generated by our approach. Our G_1 captures P_2, G_2 captures P_3, and G_3 captures P_1, respectively. An explanation graph G_b generated from an existing GNN explainer [54] captures only the pattern P_2.

Consider the following example.

Example 1: In Bitcoin blockchain transactions, money launderers employ a variety of techniques to conceal illicit cryptocurrency activities and evade detection by law enforcement agencies and AI-based monitoring systems [34, 37]. As shown in Figure 1, an input graph G includes account IP addresses and Bitcoin transactions among the accounts. Each IP address is associated with transaction-related features: the number of blockchain transactions (Txs), the number of transacted bitcoins (BTC), and the amount of fees in bitcoins (Fee). There are three targeted illicit IP addresses: v_1, v_2, and v_3. Each utilizes money laundering patterns labeled as *Spindle* [9] (P_1), *Peel Chain* [4] (P_2), and a combination (P_3) to obscure their illegal transactions. In particular, *Spindle* indicates that the perpetrators generate multiple shadow addresses to transfer illicit assets along lengthy, numerous paths that converge at a specific destination. Each path typically involves small amounts to avoid detection. The *Peel Chain* technique launders large amounts of cryptocurrency through a series of small transactions, where minor portions are 'peeled' from the original address. These incremental

outputs are typically sent to exchanges for conversion into fiat currency or other assets, minimizing detection risks due to the low value of each transfer. A GNN classifier has correctly detected all three accounts as "illicit" with associated labels.

Law enforcement agencies aim to uncover evidence that identifies these illicit IP addresses, making it essential to generate explanations that expose such activities by providing clear subgraphs that represent the money laundering patterns. For instance, an explanation graph G_b, generated by an existing GNN explanation method [54], captures pattern P_2 for node v_1 that justifies the label of v_1 as an illicit IP address. However, G_b only addresses one illicit pattern and falls short at explaining the activities associated with v_2 and v_3. In contrast, the explanations G_1, G_2, and G_3 offer a more comprehensive view, diversifying across different illicit patterns and providing distinct explanations for each targeted IP address. For example, G_1 captures the same P_2 pattern for v_1, while in G_2 the pattern P_3 identifies v_3 as illicit IP address, G_3 on the other hand, captures pattern P_1 to explain why v_2 is enlisted as illicit.

Considering multi-objective explainability measures, G_1 demonstrates the highest conciseness, as it has the smallest size among all explanatory subgraphs. In contrast, G_2 can be regarded as the most "similar" explanatory subgraph as it encompasses the majority of the nodes in G. Meanwhile, G_3 excels in its "counter" explainability, since removing it from the original graph will leave the least significant edges for classification, which are most likely to change classification to a different class. Although each of G_1, G_2, and G_3 excels in one specific criterion, achieving optimal performance across all criteria remains challenging for explanation generation. For instance, enhancing the "similar" property or diversity of explanatory subgraphs may often compromise their conciseness, due to the integration of more significant and diverse nodes. □

Can we pursue an approach that addresses all these explanatory measures? Such an explainer should (1) provide a configurable way to allow users customize GNN explanations for designated output of interests, and a set of user-centric explainability measures; (2) generate high-quality subgraph structures that simultaneously optimize multiple quality measures posed on explainability in (1); and (3) provide a diversified, comprehensive set of explanations whenever requested. The need for such "user-centric" explanations are evident for trustworthy and explainable graph-based analysis and decision making, as observed in, *e.g.,* drug repurposing [38], cybersecurity analysis [20], fraud detection [34], social recommendation [16], among other domain-specific applications.

Contribution. This paper introduces a class of configurable queries, as well as efficient algorithms, to generate diversified explanations that optimize multiple user-specified explaination quality criteria. We summarize our contribution as follows.

(1) *A formulation of Skyline Explanatory Queries.* We formalize *skyline explanatory queries* (SXQs), a class of configurable queries to clarify GNN results in terms of multiple explainability quality measures. Given a graph G, a GNN $\mathcal{M}$, nodes of interests V_T, and a set of explainability measures Φ, an SXQ aims to find explanations for the output of $\mathcal{M}$ over V_T, which simultaneously optimize all the measures in Φ.

The problem of SXQ evaluation is to generate a set of k explanations to provide a comprehensive explanation of the inference result of $\mathcal{M}$ for V_T. We formulate the problem as a multi-objective optimization problem, based on a subgraph dominance relation over multiple quality measures. We verify the hardness of both the decision and optimization version of the problem.

(2) We introduce feasible algorithms to process SXQs in terms of Pareto optimality. The algorithm adopts an "onion-peeling" strategy to iteratively reduce edges at each hop of the targeted nodes, and validate a bounded number of generated explanations in a discretized coordination system to incrementally improve the quality of the answer. We show that this process ensures a $(1 + \epsilon)$ approximation of Pareto-optimal explanation set. We also present an algorithm to diversify the answer set for SXQs.

(3) Using real-world graphs and benchmark tasks, we experimentally verify the effectiveness and scalability of our GNN explainers. We show that our approach efficiently generates a set of multi-objective optimal explanations for nodes of interest, even on large-scale graphs. For example, we outperform CF^2 in the Integrated Preference Function score by around 2.8 times on Coradataset; for OGBN_arxiv dataset with million-scale edges, we outperform the fastest baseline GNNExplainer by around 1.4 times. We also showcase of the application of our algorithms in scientific data analysis.

Related Work. We categorize related work into the following.

Graph Neural Networks. GNNs have demonstrated themselves as powerful tools in performing various graph learning tasks, including, but not limited to node classification, link prediction [58], and graph classification. [54] Recent studies proposed multiple variants of the GNNs, such as graph convolution networks (GCNs) [24], graph attention networks (GATs) [47], Graph Isomorphism Networks (GINs) [51]. These methods generally follow an information aggregation scheme where features of a target node are obtained by aggregating and combining the features of its neighboring nodes.

Explanation of GNNs. Several GNN explanation approaches have been studied. (1) *Learning-based* methods aim to learn substructures of underlying graphs that contribute to the output of a GNN. GNNExplainer [54] identifies subgraphs with node features that maximizes the influence on the prediction, by learning continuous soft masks for both adjacency matrix and feature matrix. CF-GNNExplainer [30] learns the counterfactual subgraphs, which lead to significant changes of the output if removed from the graphs. PG-Explainer [31] parameterizes the learning process of mask matrix using a multi-layer perceptron. GraphMask [41] learns to mask the edges through each layer of GNNs that leads to the most sensitive changes to their output. (2) Learning-based GNN explainers often require prior knowledge of model parameters and incur considerable learning overhead for large graphs. Other approaches perform post-processing to directly generate subgraphs that optimize a pre-assumed explanation criteria. SubgraphX [57] utilizes Monte-Carlo tree search to compute subgraphs that optimize a game-theory-inspired Shapley value. GStarX [59] follows a similar approach, yet aims to optimize "HN values", a topology-aware variant of Shapley value. These methods typically focus on optimizing a single, predefined criterion, and cannot provide a configurable mechanism for users to customize the generation of explanations.

Closer to our setting are GNN explainers that aim to optimize more than one explainability criteria. CF^2 [45] leverages both factual and counterfactual reasoning from causal inference theory to formulate a model-agnostic GNN explanation framework, which learns feature masks and edge masks aiming to produce explanations that optimize both factual and counterfactual aspects. RoboGExp [39] generates subgraph explanations that are factual, counterfactual, and meanwhile robust, *i.e.,* remains as invariant structures under a bounded number of edge modifications. GEAR [60] proposes a framework for learning GNN explainers by adjusting the gradients of multiple objectives geometrically during optimization. GEAR handles gradient conflicts globally by selecting a dominant gradient based on user-desired objectives (such as fidelity) and adjusting conflicting gradients to lie within a controlled angle threshold. MOExp [29] introduces a bi-objective optimization algorithm to find Pareto optimal explanations that strike a balance between "simulatability" (factual) and "counterfactual relevance". It proposes a zero-order search algorithm that optimizes without accessing the target GNN model's architecture or parameters, making it universally applicable. Despite these methods generating explanations that can address multiple criteria, the overall goal remains predefined – they do not provide configurable query mechanism to allow users to tune their preference as needed.

Skyline queries. Multi-objective search and skyline queries have been extensively studied [5, 12, 13]. These approaches compute Pareto optimal sets [10, 22] or their approximate variants [26, 35] over data points and a set of optimization criteria. Notable strategies include [22] that transforms multiple objectives into a single-objective counterpart. Constraint-based methods such as [10] initializes a set of anchor points that optimize each single measure, and bisects the straight lines between pairs of anchor points with a fixed vertical separation distance. This transforms bi-objective optimization into a series of single-objective counterparts. Solving each derives an approximation of the Pareto frontier. ϵ-Pareto set [26, 35] has been widely recognized as a desirable approximation for the Pareto optimal set. While these algorithms cannot be directly applied to answer SXQ, we introduce effective multi-objective optimization algorithms to generate explanatory subgraphs with provable quality guarantees in terms of ϵ-Pareto approximation.

2 GRAPHS AND GNN EXPLANATION

In this section, we provide background on graph neural networks (GNNs) and their explanations.

2.1 Graphs and Graph Neural Networks

Graphs. A directed graph $G = (V, E)$ has a set of nodes V and a set of edges $E \subseteq V \times V$. Each node v carries a tuple $T(v)$ of attributes and their values. The size of G, denoted as $|G|$, refers to the total number of its nodes and edges, *i.e.,* $|G| = |V| + |E|$.

Given a node v in G, (1) the l-hop neighbors of v, denoted as $N^l(v)$, refers to the set of nodes in the l-hop of v in G. (2) The *l-hop neighbor subgraph*, denoted as $G^l(v)$, refers to the subgraph of G induced by $N(v)$. (3) The l-hop neighbor subgraph of a set of nodes $V_s \subseteq V$, denoted as $G^l(V_s)$, refers to the subgraph induced by the node set $\bigcup_{v \in V_s} N^l(v)$.

Table 1: Summary of Preliminary Notations

Notation	Description
$G = (V, E)$	a graph G with node set V and edge set E
X, $\widetilde{\mathbf{A}}$	X: feature matrix, $\widetilde{\mathbf{A}}$: normalized adjacency matrix
$\mathcal{M}, L$	a GNN model with number of layers L
$H^{(l)}(v, G); H(v, G)$	embedding of node v in G at layer l ($l \in [1, L]$)
$\mathcal{M}(G)$, $\mathcal{M}(v, G))$	task-specific output of $\mathcal{M}$ over G and over $v \in V$
V_T	a set of (test) nodes of interests
s; G_s	a state s; a candidate G_s

Graph Neural Networks. GNNs [24] comprise a well-established family of deep learning models tailored for analyzing graph-structured data. GNNs generally employ a multi-layer message-passing scheme as shown in Equation 1.

$$\mathbf{H}^{(l+1)} = \sigma(\widetilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}) \tag{1}$$

$\mathbf{H}^{(l+1)}$ is the matrix of node representations at layer l, with $\mathbf{H}^{(0)} = \mathbf{X}$ being the input feature matrix. $\widetilde{\mathbf{A}}$ is the normalized adjacency matrix of an input graph G, which captures the topological feature of G. $\mathbf{W}^{(l)}$ is a learnable weight matrix at layer l (a.k.a "model weights"). σ is an activation function such as ReLU.

The *inference process* of a GNN $\mathcal{M}$ with L layers takes as input a graph $G = (X, \widetilde{\mathbf{A}})$, and computes the embedding $\mathbf{H}_v^{(L)}$ for each node $v \in V$, by recursively applying the update function in Equation 1. The final layer's output $\mathbf{H}^{(L)}$ (a.k.a "output embeddings") is used to generate a task-specific *output*, by applying a post-processing layer (*e.g.,* a softmax function). We denote the task-specific output as $\mathcal{M}(v, G)$, for the output of a GNN $\mathcal{M}$ at a node $v \in V$.

Fixed and Deterministic Inference. We say that a GNN $\mathcal{M}$ has a *fixed* inference process if its inference process is specified by fixed model parameters, number of layers, and message passing scheme. It has a *deterministic* inference process if $M(\cdot)$ generates the same result for the same input. We consider GNNs with fixed, deterministic inference processes. Such GNNs are desired for consistent and robust performance in practice.

Node Classification. Node classification is a fundamental task in graph analysis [17]. A GNN-based node classifier learns a GNN $\mathcal{M} : V \rightarrow \mathbf{Y}$ s.t. $\mathcal{M}(v) = y_v$ for $v \in V_{Tr} \subseteq V$, where V_{Tr} is the training set of nodes with known (true) labels Y_{Tr}. The inference process of a trained GNN $\mathcal{M}$ assigns the labels for a set of test nodes $V_T \subseteq V$, which are derived from their computed embeddings.

We summarize the frequently-used notations in Table 1.

2.2 GNN Explainers and Measures

We start with several general notations. Given a GNN $\mathcal{M}$ and an output $\mathcal{M}(v, G)$ to be explained, an *explanatory subgraph* G_ζ is an edge-induced, connected subgraph of G with a non-empty edge set $E_\zeta \subseteq E$ that are responsible to clarify the occurrence of $\mathcal{M}(v, G)$. We call the set of all explanatory subgraphs as an *interpretable domain*, denoted as ζ. A GNN*explainer* is an algorithm that generate explanatory subgraphs in ζ for an output $\mathcal{M}(v, G)$.

An explainability measure ϕ is a function: $G_\zeta \rightarrow \mathbf{R}$ that associates an explanatory subgraph to an explainability score. Given G and an output $\mathcal{M}(v, G)$ to be explained, existing GNN explainers

typically solves a single-objective optimization problem:

$$G^*_\zeta = \arg\max_{G_\zeta \in \zeta} \phi(G_\zeta) \tag{2}$$

We summarize in Table 2 a set of commonly adopted explainability measures, along with relevant GNN explainers.

3 OUR PROBLEM: SKYLINE EXPLANATIONS

We discuss the space of skyline explanatory queries for GNNs (Section 3.1), formulate our specific problem (Section 3.2), and analyze its computational complexity (Section 3.3).

3.1 Skyline Explanatory Query

Queries. We start with a class of declarative explanatory queries. A *Skyline explanatory query*, denoted as SXQ, is in the form

$$\mathsf{SXQ}(G, \mathcal{M}, V_T, \Phi)$$

where $G = (V, E)$ is an input graph, $\mathcal{M}$ is a GNN, $V_T \subseteq V$ is a set of designated test nodes of interest, and Φ is a set of user-defined explainability measures. The output $\mathcal{M}(V_T) = \bigcup_{v \in V_T}(\mathcal{M}(v, G))$ refers to the output to be explained.

Multi-objective Explanations. As aforementioned in Example 1, explanatory subgraphs that optimize a single explainability measure may not be comprehensive for the users' interpretation preference. On the other hand, a single explanatory subgraph that optimizes multiple measures may not exist, as two measures may naturally "conflict". Thus, we pursue high-quality answers for SXQ in terms of multi-objective optimality measures.

Given a node of interest $v \in V_T$, a subgraph G_ζ of G is an explanatory subgraph in the interpretable space ζ *w.r.t.* the output $\mathcal{M}(v, G)$, if it is *either* a factual *or* a counterfactual explanation. That is, G_ζ satisfies *one* of the two conditions below:

- $\mathcal{M}(v, G) = \mathcal{M}(v, G_\zeta)$;
- $\mathcal{M}(v, G) \neq \mathcal{M}(v, G \setminus G_\zeta)$

The interpretable space ζ *w.r.t.* G, $\mathcal{M}$, *and* V_T contains all the explanatory subgraphs *w.r.t.* output $\mathcal{M}(v, G)$, as v ranges over V_T.

A subset $\mathcal{G}_\zeta \subseteq \zeta$ is an *explanation w.r.t.* G, $\mathcal{M}$ and V_T if, for every node $v \in V_T$, there exists an explanatory subgraph $G_\zeta(v)$ *w.r.t.* $\mathcal{M}(v, G)$ in $\mathcal{G}$.

Explainability Measures. We make cases for three widely used explainability measures: fdl^+ measures the counterfactual property of explanatory subgraphs. Specifically, we exclude the edges of the explanatory subgraph from the original graph and conduct the GNN inference to get a new prediction based on the obtained subgraph. If the difference between these two results is significant, it indicates a good counterfactual explanatory subgraph. Similarly, fdl^- measures the factual property, i.e., how similar the explanatory subgraph is compared to the original graph in terms of getting the same predictions. conc intuitively measures how compact is the explanatory subgraph, i.e., the size of the edges.

Measurement Space. The explainability measure set Φ is a set of normalized measures to be *maximized*, each has a range $(0, 1]$. For a measure to be better minimized (*e.g.*, conciseness, fdl^- in Table 2), one can readily convert it to an inverse counterpart.

To characterize the query semantic, we introduce a dominance relation over interpretable domain ζ.

Dominance. Given a set of user-specified explanatory measures Φ (converted to *bigger* is better) and an interpretable space ζ, we say that an explanatory subgraph $G_\zeta \in \zeta$ is *dominated* by another $G'_\zeta \in \zeta$, denoted as $G_\zeta \prec G'_\zeta$, if

- for each measure $\phi \in \Phi$, $\phi(G_\zeta) \leq \phi(G'_\zeta)$; and
- there exists a measure $\phi^* \in \Phi$, such that $\phi^*(G_\zeta) < \phi^*(G'_\zeta)$.

Query Answers. We characterize the answer for an SXQ in terms of Pareto optimality. Given an interpretable space ζ *w.r.t.* G, $\mathcal{M}$, and V_T, an explanation $\mathcal{G}_\zeta \subseteq \zeta$ is a *Skyline explanation*, if

- there is no pair $\{G_1, G_2\} \subseteq \mathcal{G}_\zeta$ such that $G_1 \prec G_2$ or $G_2 \prec G_1$; and
- for any other $G \in \zeta \setminus \mathcal{G}_\zeta$, and any $G' \in \mathcal{G}_\zeta$, $G \prec G'$.

That is, $\mathcal{G}_\zeta$ is a *Pareto set* of ζ.

As a skyline explanation may still contain an excessive number of explanatory subgraphs that are too many for users to inspect, we pose a pragmatic cardinality constraint K for query answering.

A k-skyline query (denoted as SXQ^k) admits, as *query answers*, skyline explanations with at most k explanatory subgraphs, or simply *k-explanations*. Here, k is a user-defined constant ($k \leq |\zeta|$).

Example 2: Recall the example from Fig. 1. Given a skyline explanatory query $\mathsf{SXQ}(G, \mathcal{M}, V_T, \Phi)$ with $\Phi = \{\mathsf{fdl}^+, \mathsf{fdl}^-, \mathsf{conc}\}$, and $V_T = \{v_1, v_2, v_3\}$, we showcase a skyline explanation $\mathcal{G}_\zeta = \{G_1, G_2, G_3\}$ as a desired answer for the query. For example, G_1 has a relatively low fdl^+, since it contains less important edges that may affect the output of the model at v_2 and v_3; while it has the best conciseness (conc) due to its more compact representation. Meanwhile, G_2 has worse conc due to its relatively larger size; however, it has better fdl^- since it contains most of the important edges. G_3 has reasonable fdl^- and conc yet a best fdl^+ score, since it is more likely to change the output of a GNN model if removed from G, due to that it contains several most critical links to the nodes in V_T. □

3.2 Evaluation of Skyline Explanatory Queries

While one can specify Φ and k to quantify the explainability of a skyline explanation, SXQ may still return multiple explanations for users to inspect. Moreover, two k-explanations, with one dominating much less explanations than the other in ζ, may be treated "unfairly" equally good. To mitigate such bias, we adopt a natural measure to rank explanations in terms of dominance.

Given an explanatory subgraph $G_\zeta \in \zeta$, the dominance set of G_ζ, denoted as $\mathcal{D}(G_\zeta)$, refers to the largest set $\{G' | G' \prec G_\zeta\}$, *i.e.,* the set of all the explanatory subgraphs that are dominated by G_ζ in ζ. The *dominance power* of a k-explanation $\mathcal{G}_\zeta$ is defined as

$$\mathsf{DS}(\mathcal{G}_\zeta) = \left| \bigcup_{G_\zeta \in \mathcal{G}_\zeta} \mathcal{D}(G_\zeta) \right| \tag{3}$$

Note that $\mathsf{DS}(\mathcal{G}_\zeta) \leq |\zeta|$ for any explanations $\mathcal{G}_\zeta$.

Query Evaluation. Given a skyline explanatory query $\mathsf{SXQ}^k = (G, \mathcal{M}, V_T, k, \Phi)$, the query evaluation problem, denoted as $\mathsf{EVAL}(\mathsf{SXQ}^k)$, is to find a k-explanation $\mathcal{G}^{k*}_\zeta$, such that

$$\mathcal{G}^{k*}_\zeta = \arg\max_{\mathcal{G}_\zeta \subseteq \zeta, |\mathcal{G}_\zeta| \leq k} \mathsf{DS}(\mathcal{G}_\zeta)$$

Table 2: Common explainability measures and notable GNN explainers. Here $|\mathcal{G}_\zeta|$ refers to the total number of edges in the explanatory subgraphs in $\mathcal{G}_\zeta$. In particular, [29, 45] find explanations that are both factual and counterfactual.

Symbol	Measure	Equation	Range	Description	Explainers
fac	factual	$\mathcal{M}(v,G) = \mathcal{M}(v,G_\zeta)$?	{true,false }	a Boolean function	[29, 31, 45, 54]
cfac	counterfactual	$\mathcal{M}(v,G) \neq \mathcal{M}(v,G \setminus G_\zeta)$?	{true,false }	a Boolean function	[29, 30, 45]
fdl^+ [56]	fidelity^+	$Pr(M(v,G)) - Pr(M(v,G \setminus G_\zeta))$	$[-1,1]$	the larger, the better	[8, 57]
fdl^- [56]	fidelity^-	$Pr(M(v,G)) - Pr(M(v,G_\zeta))$	$[-1,1]$	the smaller, the better	[8, 31]
conc [56]	conciseness	$\frac{1}{N}\sum_{i=1}^{N}(1 - \frac{\lvert E(G_\zeta)\rvert}{\lvert E(G)\rvert})$	$[0,1]$	the smaller, the better	[29, 57]
shapley [25]	Shapley value	$\phi(G_\zeta) = \sum_{S \subseteq P \setminus \{G_\zeta\}} \frac{\lvert S\rvert!(\lvert P\rvert-\lvert S\rvert-1)!}{\lvert P\rvert!} m(S,G_\zeta)$	$[-1,1]$	Total contribution of nodes in G_ζ	[57]

3.3 Computational Complexity

We next investigate the hardness of evaluating skyline exploratory queries. To this end, we start with a verification problem.

Verification of Explanations. Given a query SXQ = $(G, \mathcal{M}, V_T, \Phi)$, and a set of subgraphs $\mathcal{G}$ of G, the verification problem is to decide if $\mathcal{G}$ is an explanation.

Theorem 1: *The verification problem for* SXQ *is in P.* □

Proof sketch: Given an SXQ = $(G, \mathcal{M}, V_T, k, \Phi)$ and a set of subgraphs $\mathcal{G}$ of G, we provide a procedure, denoted as Verify, that correctly determines if $\mathcal{G}$ is an explanation. The algorithm checks, for each pair (v, G_s) with $v \in V_T$ and $G_s \in \mathcal{G}$, if G_s is a factual or a counterfactual explanation of $\mathcal{M}(v,G)$. It has been verified that this process can be performed in PTIME, by invoking a polynomial time inference process of $\mathcal{M}$ for v over G_s (for testing factual explanation) and $G \setminus G_s$ (for testing counterfactual explanations), respectively [7, 39]. It outputs true if there exists a factual or counterfactual explanation for every $v \in V_T$; and false otherwise. □

While it is tractable to verify explanations, the evaluation of an SXQ is already nontrival for $|\Phi| = 3$, even for a constrained case that $|\zeta|$ is a polynomial of $|G|$, *i.e.,* there are polynomially many connected subgraphs to be explored in G.

Theorem 2: $\text{EVAL}(\text{SXQ}^k)$ *is* NP*-hard even when* $|\Phi| = 3$ *and* $|\zeta|$ *is polynomially bounded by* $|G|$. □

Proof sketch: The hardness of the problem can be verified by constructing a polynomial time reduction from k-representative skyline selection problem (k-RSP) [28]. Given a set S of data points, the problem is to compute a k-subset S^* of S, such that (a) S^* is a Pareto-set, and (b) S^* maximizes the dominance score DS.

Given an instance of k-RSP with a set S, we construct an instance of $\text{EVAL}(\text{SXQ}^k)$ as follows. (1) For each data point $s \in S$, create a node v_s, and construct a distinct, single-edge tree T_s at v_s. Assign a ground truth label l_s to each v_s. Let G be the union of all the single-edge trees, and define V_T as the set of root nodes of all such trees. (2) Duplicate the above set G' as a training graph G_T and train a GNN classifier $\mathcal{M}$ with layer $L \geq 1$, which gives the correct outputs. For mainstream GNNs, the training cost is in PTIME [7]. Set Φ to be a set of functions, where each $\phi \in \Phi$ assigns the i-th value of a data point s in the instance of k-RSP to be the value for the i-th explanatory measure of the matching node v_s, where $i \in [1, d]$ for d-dimensional data point in k-RPS problem. (3) Apply $\mathcal{M}$ to G with V_T as test set. Given that $\mathcal{M}$ is fixed and deterministic, the inference ensures the invariance property [18] (which generates the same results for isomorphic input G and G_T). That is, $\mathcal{M}$ assigns consistently and correctly the ground truth labels to each node v_s in G. Recall that the explanatory subgraph is connected with a non-empty edge set (§ 2.2). This ensures that T_s is the only factual explanation for each $v_s \in V_T$ in G. Each T_s may vary in Φ.

As $|\zeta|$ is in $O(f(|G|))$ for a polynomial function f, the above reduction is in PTIME. We can then show that there exists a k representative skyline set for k-RSP, if and only if there exists a k-explanation as an answer for the constructed instance of $\text{EVAL}(\text{SXQ}^k)$. As k-RSP is NP-hard for 3-dimensional space with a known input dataset, $\text{EVAL}(\text{SXQ}^k)$ remains NP-hard for the case that $|\zeta|$ is polynomially bounded by $|G|$, and $|\Phi|$=3. □

An Exact Algorithm. A straightforward algorithm evaluates SXQ^k with exact optimal explanation. The algorithm first induces a subgraph G^L with edges within L-hop neighbors of the nodes in V_T, where L is the number of layers of the GNN $\mathcal{M}$. It then initializes the interpretable space ζ as all *connected* subgraphs in G^L. This can be performed by invoking subgraph enumeration algorithms [23, 53]. It then enumerates n skylines from ζ and finds an optimal k-explanation.

The above algorithm correctly evaluates SXQ with optimal explanations for GNNs, yet is not practical for large V_T and G, as n alone can be already 2^{deg^L} (the number of connected subgraphs in G_L), and the Pareto sets to be inspected can be $\binom{n}{k}$.

We thus resort to approximate query processing for SXQ, and present efficient algorithms that do not require enumeration to generate explanations that approximate optimal answers.

4 GENERATING SKYLINE EXPLANATIONS

4.1 Approximating Skyline Explanations

We introduce our first algorithm to approximately evaluate a skyline explanatory query. To characterize the quality of the answer, we introduce a notion of ϵ-explanation.

ϵ-explanations. Given explanatory measures Φ and an interpretable space ζ, we say that an explanatory subgraph $G_\zeta \in \zeta$ is *ϵ-dominated* by another $G'_\zeta \in \zeta$, denoted as $G_\zeta \preceq_\epsilon G'_\zeta$, if

- for each measure $\phi \in \Phi$, $\phi(G_\zeta) \leq (1+\epsilon)\phi(G'_\zeta)$; and
- there exists a measure $\phi^* \in \Phi$, such that $\phi^*(G_\zeta) \leq \phi^*(G'_\zeta)$.

Given a k-skyline query $\text{SXQ}^k = (G, \mathcal{M}, V_T, \Phi)$ and an interpretable domain ζ *w.r.t.* G, $\mathcal{M}$, and V_T, an explanation $\mathcal{G}_\epsilon \subseteq \zeta$ is an *(ζ,ϵ)-explanation w.r.t.* G, $\mathcal{M}$, and V_T, if (1) $|\mathcal{G}_\epsilon| \leq k$, and (2) for any

explanatory subgraph $G_\zeta \in \zeta$, there is an explanatory subgraph $G'_\zeta \in \mathcal{G}_\epsilon$, such that $G_\zeta \preceq_\epsilon G'_\zeta$.

For a k-skyline query SXQ^k, a (ζ, ϵ)-explanation $\mathcal{G}_\epsilon$ properly approximates a k-explanation $\mathcal{G}_\zeta$ as its answer in the interpretable domain ζ. Indeed, (1) $\mathcal{G}_\epsilon$ has a bounded number k of explanatory subgraphs as $\mathcal{G}_\zeta$. (2) $\mathcal{G}_\epsilon$ is, by definition, an ϵ-Pareto set of ζ. In multi-objective decision making, an ϵ-Pareto set has been an established notion as a proper size-bounded approximation for a Pareto optimal set (a k-explanation $\mathcal{G}_\zeta$, in our context) [42].

(α, ϵ)-Approximations. Given a k-skyline query SXQ^k $(G, \mathcal{M}, V_T, \Phi)$, and an interpretable domain ζ *w.r.t.* G, $\mathcal{M}$ and V_T, let $\mathcal{G}^*_\zeta$ be the optimal k-explanation answer for SXQ^k in ζ (see Section 3.2). We say that an algorithm is an (α, ϵ)-*approximation* for the problem $\mathsf{EVAL}(\mathsf{SXQ}^k)$ *w.r.t.* ζ, if it ensures the following:

- it correctly computes an (ζ, ϵ)-explanation $\mathcal{G}_\epsilon$;
- $\mathsf{DS}(\mathcal{G}_\epsilon) \geq \alpha \mathsf{DS}(\mathcal{G}^*_\zeta)$; and
- it takes time in $O(f(|\zeta|, |G|, \frac{1}{\epsilon}))$, where f is a polynomial.

We present our main result below.

Theorem 3: *There is a $(\frac{1}{4}, \epsilon)$-approximation for* $\mathsf{EVAL}(\mathsf{SXQ}^k)$ w.r.t. *ζ', where ζ' is the set of explanatory subgraphs verified by the algorithm. The algorithm computes a (ζ', ϵ)-explanation in time* $O(|\zeta'|(\log \frac{r_\Phi}{\epsilon})^{|\Phi|} + L|G^L(V_T)|)$. □

Here (1) $r_\Phi = \max \frac{\phi_u}{\phi_l}$, for each measure $\phi \in \Phi$ with a range $[\phi_l, \phi_u]$; (2) $G^L(V_T)$ refers to the set of all the L-hop neighbor subgraphs of the nodes V_T, and (3) L the number of layers of the GNN $\mathcal{M}$. Note that in practice, $|\Phi|$, L and $\epsilon \in [0, 1]$ are small constants, and the value of r_Φ is often small.

4.2 Approximation Algorithm

As a constructive proof of Theorem 3, we first introduce an approximation algorithm for $\mathsf{EVAL}(\mathsf{SXQ}^k)$, denoted as ApxSX-OP.

The algorithm ApxSX-OP takes advantage of a data locality property: For a GNN $\mathcal{M}$ with L layers and any node v in G, its inference computes $\mathcal{M}(v, G)$ with up to L-hop neighbors of v ($N^L(v)$) via message passing, regardless of how large G is. Hence, it suffices to explore and verify connected subgraphs in the L-hop neighbor subgraph $G^L(V_T)$ (see Section 2).

In a nutshell, it defines three procedures and their interactions:

(1) a Generator, which initializes and continuously explores a potential interpretable domain ζ' by generating a sequence ("a stream") of candidate explanatory subgraphs (or simply a "candidate") from $G^L(V_T)$;

(2) a Verifier, which asserts if an input candidate G_s is an explanatory subgraph for SXQ^k; and

(3) an Updater, that dynamically maintains a current size k (ζ', ϵ)- explanation $\mathcal{G}_\epsilon$ over verified candidates ζ', upon the arrival of verified explanatory subgraph in (2), along with other auxiliary data structures. The currently maintained explanation $\mathcal{G}_\epsilon$ is returned either upon termination (to be discussed in following parts), or upon an ad-hoc requests at any time from the queryer.

Auxiliary structures. ApxSX-OP dynamically maintains the following structures to coordinate the procedures.

Algorithm 1 ApxSX-OP Algorithm

Input: a query $\mathsf{SXQ}^k = (G, \mathcal{M}, V_T, \Phi)$; a constant $\epsilon \in [0, 1]$;
Output: a (ζ', ϵ)-explanation $\mathcal{G}_\epsilon$.

```
1:  set G_ε := ∅;
2:  identify edges for each hop: E={E_L, E_{L-1}, ..., E_1};
3:  for l = L to 1 do                          # Generator: Onion Peeling
4:     initializes state s_0 = G^l(V_T) (l-hop neighbor subgraph).
5:     while E_l ≠ ∅ do
6:        for e ∈ E_l do
7:           spawns a state s with a candidate G_s := G^l\{e};
8:           update ζ' with state s and a new transaction t;
9:           if vrfyF(s)=False & vrfyCF(s)=False then    # Verifier
10:             continue;
11:          G_ε :=updateSX (s, G_s, ζ', G_ε); E_l=E_l\{e};    # Updater
12: return G_ε.
```

State Graph. In particular, ApxSX-OP interacts with the Generator, Verifier and Updater via a state graph (simply denoted as ζ'). Each node (a "state") $s \in \zeta'$ records a candidate G_s and its local information to be updated and used for evaluating SXQ^k. There is a directed edge (a "transaction") $t = (s, s')$ in ζ', if $G_{s'}$ is obtained by applying a graph editing operator (*e.g.*, edge insertion, edge deletion) to G_s. A path ρ in the state graph ζ' consists of a sequence of transactions that results in a candidate.

In addition, each state s is associated with (1) a score $\mathsf{DS}(G_s)$, (2) a coordinate $\Phi(s)$, where each entry records an explainability measure $\phi(G_s)$ ($\phi \in \Phi$); and (3) a variable-length bitvector $B(s)$, where an entry $B(s)[i]$ is set as 1 if $G_i \preceq_\epsilon G_s$, and 0 otherwise. Note that the vector $B(s)$ bookkeeps the ϵ-dominance relation between G_s and the rest of candidates in ζ'. Its $\mathsf{DS}(G_s)$ score over ζ', including all the verified candidates so far, can be easily counted as the number of "1" entries in $B(s)$.

"Onion Peeling". To reduce unnecessary verification, ApxSX-OP adopts a prioritized edge deletion strategy called "onion peeling". Given a node v and its L-hop neighbor subgraph $G^L(v)$, it starts with an initial state s_0 that corresponds to $G^L(V_T)$, and iteratively removes edges from the "outmost" L-th hop "inwards" to v (via Generator). This spawns a set of new candidates to be verified (by invoking Verifier), and the explanatory subgraphs are processed by Updater to maintain the explanation.

This strategy enables several advantages. (1) Observe that $\mathcal{M}(G^L(v), v) = \mathcal{M}(G, v)$ due to data locality. Intuitively, it is more likely to discover explanatory subgraphs earlier, by starting from candidates with small difference to $G^L(v)$, which is by itself a factual explanatory subgraph. (2) The strategy fully exploits the connectivity of $G^L(v)$ to ensure that the Generator produces only connected candidates with v included, over which DS, Φ and dominance are well defined. In addition, the process enables early detection and skipping of non-dominating candidates (see "Optimization").

Algorithm. Algorithm ApxSX-OP dynamically maintains ζ' as the state graph. It first induces and verifies the L-hop neighbor subgraph $G^L(V_T)$, and initializes state node s_0 (w.r.t G_{s_0}) with $G^L(V_T)$ and local information. It then induces L-batches of edge sets E_i $i \in [1, L]$ from $G^L(V_T)$ for onion peeling processing. For each "layer" (line 3)

Procedure 2 Procedure updateSX

Input: a state s, an explanatory subgraph G_s, state graph ζ'; an explanation $\mathcal{G}_\epsilon$;
Output: updated (ζ', ϵ)-explanation $\mathcal{G}_\epsilon$;

1: initializes state s with structures DS(s), $B(s)$:=$\emptyset$, $\Phi(G_s)$:=$\emptyset$;
2: evaluates $\Phi(G_s)$;
3: incrementally determines $(1+\epsilon)$-dominance of G_s;
4: updates $B(s)$ and DS(s);
5: **if** $\{G_s\}$ is a new skyline explanation **then** # New skyline
6: **if** $|\mathcal{G}_\epsilon|<k$ **then**
7: $\mathcal{G}_\epsilon := \mathcal{G}_\epsilon \cup \{G_s\}$;
8: **else**
9: $\mathcal{G}_\epsilon$:= swap $(\mathcal{G}_\epsilon, s)$;
10: **return** $\mathcal{G}_\epsilon$.

(E_l, $1 \le l \le L$), the Generator procedure iteratively selects a next edge e to be removed from the current layer and generates a new candidate $G_{s'}$ by removing e from G_s, spawning a new state s' in ζ' with a new transaction $t = (s, s')$. Following this procedure, we obtain a "stream" of states to be verified. Each candidate is then processed by the Verifier procedures, vrfyF and vrfyCF, to test if G_s is factual or counterfactual, respectively (lines 9-10). If G_s passes the test, the associated state $s \in \zeta'$ is processed by invoking the Updater procedure updateSX, in which the coordinator $\Phi(s)$, $(1+\epsilon)$-dominance relation (encoded in $B(s)$), and DS(s) are incrementally updated (line 1). updateSX then incrementally maintains the current explanation $\mathcal{G}_\epsilon$ with the newly verified explanatory subgraph G_s, following a replacement strategy (see Procedure updateSX). The processed edge e is then removed from E_l (line 11).

Example 3: Consider the example in Fig. 2. Algorithm ApxSX-OP start the generation of explanatory subgraphs within the 2-hop subgraph s_0, by deleting one edge from the set of hop-2 edges, i.e., e_1, e_2, and e_3. This spawns three states s_1, s_2, and s_3 to be verified and evaluated to maintain a skyline explanation. It chooses s_3 as the next state to explore, which leads to the states s_4 and e_5, in response to the deletion of e_1 and e_2, respectively. It continues to verify states s_4 and s_5. As s_4 fails the verification, i.e., it is not factual or counterfactual, it continues to verify s_5. This gives a current answer set that contains $\{s_3, s_5\}$. □

Procedure updateSX. For each new explanatory subgraph G_s (at state s), updateSX update its information by (1) computing coordinate $\Phi(G_s)$, (2) incrementally determines if G_s is likely to join a skyline explanation in terms of $(1+\epsilon)$-dominance, *i.e.,* if for any verified explanatory subgraph $G_{s'}$ in ζ', $G_{s'} \preceq_\epsilon G_s$ (to be discussed). If so, and if the current explanation $\mathcal{G}_\epsilon$ has a size smaller than k, G_s is directly added to $\mathcal{G}_\epsilon$. (line 5- 7). Otherwise, updateSX performs a swap operator as follows: 1) identify the skyline explanation $\bar{s} \in \mathcal{G}_\epsilon$ that has the smallest $\mathcal{D}(\bar{s})$; 2) replace $\bar{s}$ with G_s, only when such replacement makes the new explanation $\mathcal{G}'_\epsilon$ having a higher score $\mathcal{D}(\mathcal{G}'_\epsilon)$ increased by a factor of $\frac{1}{k}$ (line 8- 9).

Update Skyline Explanations. Procedure updateSX maintains a set of skyline explanations $\mathcal{G}_s$ (not shown) in terms of $(1+\epsilon)$-dominance. The set $\mathcal{G}_s$ is efficiently derived from the bitvectors $B(s)$

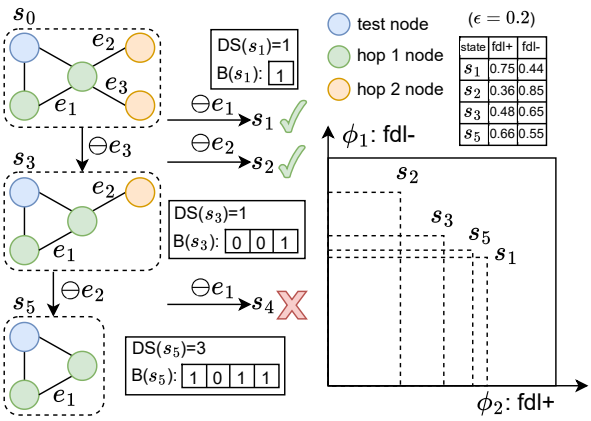


Figure 2: Illustration of Onion Peeling (L=2, k=2). $\zeta' = \{s_1, s_2, s_3, s_5\}$. A (ζ', ϵ)-explanation $\mathcal{G}_\epsilon = \{s_2, s_5\}$ with DS = 3.

of the states in ζ'. The latter compactly encodes a lattice structure of $(1+\epsilon)$ dominance as a directed acyclic graph; and $\mathcal{G}_s$ refers to the states with no "parent" in the lattice, *i.e.,* having an explanatory subgraph that is not $(1+\epsilon)$-dominated by any others so far. We present details in anonymous full version [**?**].

Example 4: Recall Fig. 2 where the explanation space $\zeta' = \{s_1, s_2, s_3, s_5\}$, the $(1+\epsilon)$-dominance table is shown on the top-right. As we explore the explanatory subgraph in an order (stream) of s_1, s_2, s_3, s_5, we can see that the first two states will be skylines and added into the explanation set. When s_3 comes in, since the improvement is less than $\frac{3}{2}$, i.e., $(1+\frac{1}{k})$, we will pass s_3. Then for s_5, since it $(1+\epsilon)$-dominates s_1 and s_3, it will replace s_1 and the final skyline explanation will be $\{s_2, s_5\}$. □

Quality Guarantee. Algorithm ApxSX-OP always terminates as it constantly removes edges from $G^L(V_T)$ to explore and verify a finite set of candidates. To see the quality guarantees and approximability, we show two results below.

Lemma 4: *Given a constant ϵ,* ApxSX-OP *correctly computes a (ζ', ϵ)-explanation of size k defined on the interpretation domain ζ', which contains all verified candidates.* □

Proof sketch: We show the above result with a reduction to the multi-objective shortest path problem (MOSP) [46]. Given an edge-weighted graph G_w, where each edge carries a d-dimensional attribute vector $e_w.c$, it computes a Skyline set of paths from a start node u. The cost of a path ρ_w in G_w is defined as $\rho_w.c = \sum_{e_w \in \rho_w} e_w.c$. The dominance relation between two paths is determined by the dominance relation of their cost vector. Our reduction (1) constructs G_w as the running graph ζ' with n verified states and transitions; and (2) for each edge (s, s'), sets an edge weight as $e_w = \Phi(s) - \Phi(s')$. Given a solution Π_w of the above instance of MOSP, for each path $\rho_w \in \Pi$, we set a corresponding path ρ in ζ' that ends at a state ρ_s , and adds it into $\mathcal{G}_\epsilon$.

We can verify that Π_w is an ϵ-Skyline set of paths Π_w in G_w, if and only if $\mathcal{G}_\epsilon$ is an (ζ', ϵ)-explanation of ζ'. We show that ApxSX-OP performs a more simpler process of the algorithm in [46], which ensures to generate $\mathcal{G}_\epsilon$ as a (ζ', ϵ)-explanation. □

Lemma 5: ApxSX-OP *correctly computes a* (ζ', ϵ)*-explanation* $\mathcal{G}_\epsilon$ *that ensures* $\mathsf{DS}(\mathcal{G}_\epsilon) \geq \frac{1}{4}\mathsf{DS}(\mathcal{G}^*_\epsilon)$*, where* $\mathcal{G}^*_\epsilon$ *is the size* k (ζ', ϵ)*-explanation over* ζ' *with maximum dominance power* $\mathsf{DS}(\mathcal{G}^*_\epsilon)$. □

Proof sketch: Consider procedure updateSX upon the arrival, at any time, of a new verified candidate G_s. (1) The above results clearly hold when $|\zeta'| \leq k$ or $|\mathcal{G}_\epsilon| \leq k$, as $\mathcal{G}_\epsilon$ is the only (ζ', ϵ)-explanation so far. (2) When $|\mathcal{G}_\epsilon| = k$, we reduce the approximate evaluation of SXQ^k to an instance of the *online MAX k-SET COVERAGE* problem [2]. The problem maintains a size-k set cover with maximized weights. We show that updateSX adopts a greedy replacement policy by replacing a candidate in $\mathcal{G}_\epsilon$ with the new candidate G_s only when this leads to a $(1 + \frac{1}{k})$ factor improvement for DS. This ensures a $\frac{1}{4}$-approximation ratio [2]. Following the replacement policy consistently, updateSX ensures a $\frac{1}{4}$ approximation ratio for (ζ', ϵ)-explanations in ζ'. □

Time Cost. As Algorithm ApxSX-OP follows an online processing for candidates generated from a stream, we provide its time cost in an output sensitive manner, in terms of the interpretation domain ζ' when terminates. ApxSX-OP first induces the L-hop neighbor subgraph of V_T, in $O(|G^L(V_T)|)$ time. ApxSX-OP process in total $|\zeta'|$ candidates. For each candidate G_s, the Verifier process verifies G_s for SXQ (procedures vrfyF and vrfyCF), which incurs two inferences of GNN $\mathcal{M}$, in total $O(L|G^L(V_T)|)$ (assuming the number of node features are small; *cf.* Lemma 1, and [7, 39]. The total verification cost is thus in $O(|\zeta'|L|G|)$ time. For each verified explanatory subgraph G_s, the Updater procedure updateSX follows a simplified process to update a set of $(1 + \epsilon)$ skyline set in ζ' by solving a multi-objective dominating paths problem in the state graph ζ', which takes at most $\prod_{i=1}^{|\Phi|}\left(\left\lfloor \log_{1+\epsilon}\frac{\phi_u}{\phi_l}\right\rfloor + 1\right)$) time. Here ϕ_l and ϕ_u refers to the minimum and maximum value an explainability measure $\phi \in \Phi$ has in the verified candidates in ζ'. Given ϵ is small, $\log(1 + \epsilon) \approx \epsilon$, hence the total maintenance cost is in $O\left(|\zeta'| \cdot \left(\frac{\log(r_\Phi)}{\epsilon}\right)^{|\Phi|}\right)$ time, with $r_\Phi = \frac{\phi_u}{\phi_l}$. Putting these together, the total cost is in $O(|\zeta'|(\log \frac{r_\Phi}{\epsilon})^{|\Phi|} + L|G^L(V_T)|)$ time.

Putting the above analysis together, Theorem 3 follows.

Optimization. To further reduce verification cost, ApxSX-OP uses two optimization strategies, as outlined below.

Candidate Prioritization. ApxSX-OP adopts an edge prioritization heuristic to favor promising candidates with small loss of DS and that are more likely to be a skyline explanation (Algorithm ApxSX-OP, line 7). It ranks each transaction t=(s, s') in ζ based on a loss estimation of DS by estimating $\Phi(s')$. The cost vector of each transaction is aggregated to an average weight based on each dimension, *i.e.,* $w(t) = \frac{1}{|\Phi|}\sum_{i=1}^{|\Phi|}(\phi_i(s) - \phi_i(\hat{s}'))$. The candidates from spawned states with a smallest loss of DS are preferred. This helps early convergence of high-quality explanations, and also promotes early detection of non-dominating candidates (see "early pruning").

Early Pruning. ApxSX-OP also exploits a *monotonicity property* of measures $\phi \in \Phi$ to early determine the non-ϵ-dominance of an explanatory subgraph, even before its measures are computed. Given ζ' and a measure $\phi \in \Phi$, we say ϕ is *monotonic w.r.t.* a path ρ in ζ', if

Procedure 3 Procedure updateDivSX

Input: a state s, an explanatory subgraph G_s, state graph ζ'; an explanation $\mathcal{G}_\epsilon$;
Output: Updated (ζ', ϵ)-explanation $\mathcal{G}_\epsilon$;

```
initializes state s with structure Φ(G_s):=∅;
evaluates Φ(G_s);
incrementally determines (1+ε)-dominance of G_s;
if {G_s} is a new skyline explanation then        # New skyline
    Δ(s|G_ε) := DivS(G_ε ∪ {G_s}) − DivS(G_ε)
    if |G_ε|<k and Δ(s|G_ε) ≥ ((1+ε)/2 − DivS(G_ε)) / (k − |G_ε|) then
        G_ε := G_ε ∪ {G_s};
return G_ε.
```

for a state s with candidate G_s and another state s' with a subgraph $G_{s'}$ of G_s on the same path ρ, $\phi_l(G_s) \geq \frac{\phi_u(G_{s'})}{1+\epsilon}$, where $\phi_l(G_s)$ (resp. $\phi_u(G_{s'})$) is a lower bound estimation of $\phi(G_s)$ (resp. upper bound estimation of $\phi(G_{s'})$), *i.e.,* $\phi_l \leq \phi_l(G_s) \leq \phi(G_s)$ (resp. $\phi(G_{s'}) \leq \phi_u(G_{s'}) \leq \phi_u$). By definition, for any ϕ with monotonicity property, we have $\phi(G_{s'}) \leq \phi_u(G_{s'}) \leq (1 + \epsilon)\phi_l(G_s) \leq (1 + \epsilon)\phi(G_s)$, hence $G_{s'} \preceq_\epsilon G_s$, and any such subgraphs $G_{s'}$ of G_s can be safely pruned due to non-$(1 + \epsilon)$ dominance determined by ϕ alone, without further verification. Explainability measures such as density, total influence, conciseness, and diversity of embeddings, are likely to be monotonic. Hence, the onion peeling strategy enables early pruning by exploiting the inherent or estimated ranges of such measures. Note that the above property is checkable in $O(|\zeta'|)$ time.

Alternative Strategy: Edge Growing. As the end user may want to early termination and obtain compact, smaller-sized explanations, we also outline a variant of ApxSX-OP, denoted as ApxSX-I. It follows the same Verifier and Updater procedures, yet uses a different Generator procedure that starts with a single node v and inserts edges to grow candidate, level by level, up to its L-hop neighbor subgraph. ApxSX-I remains to be an $(\frac{1}{4}, \epsilon)$-approximation for $\mathsf{EVAL}(\mathsf{SXQ}^k)$, and does not incur additional time cost compared with ApxSX-OP. We present detailed analysis in [**?**].

5 DIVERSIFIED SKYLINE EXPLANATIONS

Skyline explanation may still contain explanatory subgraphs having many similar nodes. This may lead to redundant and biased explanations, as remarked earlier. Intuitively, one also prefers the explanatory subgraphs to contain various types of nodes that can clarify the model output with a more comprehensive view when inspected. We next introduce an algorithm that can compute diversified answers for SXQ queries.

Diversification function. Given a query $\mathsf{SXQ}^k = (G, \mathcal{M}, V_T, k, \Phi)$, the diversified evaluation problem, denoted as $\mathsf{DivEVAL}(\mathsf{SXQ}^k)$, is to find a (ζ', ϵ)-explanation $\mathcal{G}_\epsilon$, such that

$$\mathcal{G}^*_\epsilon = \underset{\mathcal{G} \subseteq \zeta, |\mathcal{G}_\epsilon| \leq k}{\arg\max} \ \mathsf{DivS}(\mathcal{G}_\epsilon)$$

where $\mathsf{DivS}(\mathcal{G}_\epsilon)$ is a *diversification function* defined on $\mathcal{G}_\epsilon$, to quantify its overall diversity. Below we introduce such a function.

As remarked earlier, we quantify the diversity of an explanation as a bi-criteria function, in terms of both neighborhood coverage

and the difference between node representations, and is defined as

$$\mathsf{DivS}(\mathcal{G}_\epsilon) = \alpha \cdot \mathsf{NCS}(\mathcal{G}_\epsilon) + (1-\alpha) \cdot \sum_{G_s, G_{s'} \in \mathcal{G}_\epsilon} \mathsf{CD}(G_s, G_{s'}) \quad (4)$$

(1) NCS, a *node coverage* measure, aggregates the node coverage of explanatory subgraphs in $\mathcal{G}_\epsilon$, and (2) CD, an *accumulated difference* measure, aggregates the difference between two explanatory subgraphs. The two terms are balanced by a constant α.

Specifically, we adopt a simple node coverage function as

$$\mathsf{NCS}(\mathcal{G}_\epsilon) = \frac{|\bigcup_{G_s \in \mathcal{G}_\epsilon} V_{G_s}|}{|V_{G^L}|} \quad (5)$$

and for graph differences, we define CD as the accumulated Cosine distances between two explanatory graphs as:

$$\mathsf{CD}(G_s, G_{s'}) = 1 - \frac{\mathbf{x}_{G_s} \cdot \mathbf{x}_{G_{s'}}}{||\mathbf{x}_{G_s}||_2 \cdot ||\mathbf{x}_{G_{s'}}||_2} \quad (6)$$

Here, $\mathbf{x}_{G_s}$ is the embedding of G_s obtained by graph representation models such as Node2Vec [19].

Diversification Algorithm. We next introduce our diversified evaluation algorithm, denoted as DivSX. It follows ApxSX-OP and adopts onion peeling strategy to generate subgraphs in a stream. The difference is that when computing the k-skyline explanation, it includes a new replacement strategy when the marginal gain for the diversification function $\mathsf{DivS}(\cdot)$ is at least a factor of the score over the current solution $\mathsf{DivS}(\mathcal{D}_\epsilon)$. DivSX terminates when a first k-skyline explanation is found.

Procedure updateDivSX. For each new candidate G_s (state s), updateDivSX first initializes and updates the candidate G_s by (1) computing its coordinate $\Phi(G_s)$ (line 2); (2) incrementally determines if G_s is a skyline explanation in terms of $(1+\epsilon)$-dominance, *i.e.,* if for any verified explanatory subgraph $G_{s'}$ in ζ', $G_{s'} \preceq_\epsilon G_s$ (line 3). If so, 1) check if the current explanation $\mathcal{G}_\epsilon$ has a size smaller than k; 2) the marginal gain of G_s is bigger than $\frac{(1+\epsilon)/2 - \mathsf{DivS}(\mathcal{G}_\epsilon)}{k - |\mathcal{G}_\epsilon|}$ (line 5- 6). If the skyline satisfies the above two conditions, include it in the k-skyline explanation $\mathcal{G}_\epsilon$ (line 7).

Quality and Approximability Guarantee. Algorithm DivSX always terminates as it constantly removes edges from $G^L(V_T)$ to explore and verify a finite set of candidates. To see the quality guarantees and approximability, we show two results below.

Lemma 6: *Given a constant ϵ,* DivSX *correctly computes a (ζ', ϵ)-explanation of size k defined on the interpretation domain ζ', which contains all verified candidates.* □

The proof is similar to Lemma 4, therefore we omit it here.

Theorem 7: DivSX *correctly computes a (ζ', ϵ)-explanation $\mathcal{G}_\epsilon$ that ensures* $\mathsf{DivS}(\mathcal{G}_\epsilon) \geq (\frac{1}{2} - \epsilon)\mathsf{DivS}(\mathcal{G}^*_\epsilon)$, *where $\mathcal{G}^*_\epsilon$ is the size k (ζ', ϵ)-explanation over ζ' with maximum diversity power* $\mathsf{DivS}(\mathcal{G}^*_\epsilon)$. □

Proof sketch: Both $\mathsf{NCS}(\cdot)$ and $\mathsf{CD}(\cdot)$ are submodular functions. For $\mathsf{NCS}(\cdot)$, since we only modify the presence of edges and keep the nodes unchanged, when a new skyline is included in the k-skyline explanation, the number of covered nodes will increase or remain the same. For $\mathsf{CD}(\cdot)$, when a new skyline is included, the number of pair-wise cosine distance computations will increase by $|\mathcal{G}_\epsilon|$, therefore the overall $\mathsf{CD}(\cdot)$ will increase or remain the same.

Table 3: Statistics of datasets

dataset	# nodes	# edges	# node features	# class labels
Cora	2,708	10,556	1,433	7
PubMed	19,717	88,648	500	3
FacebookPage	22,470	342,004	128	4
AmazonComputer	13,752	491,722	767	10
OGBN_arxiv	169,343	1,166,243	128	40

Consider procedure updateDivSX upon the arrival, at any time, of a new verified candidate G_s. Given that $|\mathcal{G}_\epsilon| \leq k$ is a hard constraint, we reduce the approximate diversified evaluation of SXQ^k to an instance of the *Streaming Submodular Maximization* problem [3]. The problem maintains a size-k set with maximized submodular function value over a stream of data objects. We show that DivSX adopts a greedy increment policy by including a candidate in $\mathcal{G}_\epsilon$ with the new candidate G_s only when this leads to a marginal gain greater than $\frac{(1+\epsilon)/2 - f(S)}{k - |S|}$, where S is the current set and $f(\cdot)$ is a submodular function. This ensures a $(\frac{1}{2} - \epsilon)$-approximation ratio [3]. Following the increment policy consistently, DivSX ensures a $(\frac{1}{2} - \epsilon)$-approximation ratio for (ζ', ϵ)-explanations in ζ'. □

Time Cost. Since Algorithm DivSX follows the same process as ApxSX-OP, and the update time of DivSX is $O(1)$ [3]. Therefore, according to ApxSX-OP, the time cost of DivSX is also $O(|\zeta'|(\log \frac{r_\Phi}{\epsilon})^{|\Phi|} + L|G^L(V_T)|)$.

6 EXPERIMENTAL STUDY

We conduct experiments to evaluate the effectiveness, efficiency, and scalability of our solutions. Our algorithms are implemented in Python 3.10.14 by PyTorch-Geometric framework. All experiments are conducted on a Linux system equipped with AMD Ryzen 9 5950X CPU, an NVIDIA GeForce RTX 3090, and 32 GB of RAM. Our code and data are made available at [1].

6.1 Experimental Setup

Datasets. We used Cora [32], PubMed [43], FacebookPage [40], AmazonComputer [44], and OGBN_arxiv [21] (Table 3).

(1) Both Cora and PubMed are citation networks. The nodes represent papers, and edges denote citation relation between papers. Each node has a feature encoding that corresponds to the presence or absence of a keyword from a dictionary. For both, we consider a node classification task that assigns a topic area to each node.

(2) For FacebookPage, the nodes represent verified Facebook pages, and edges are mutual "likes". The node features are extracted from the site descriptions. The task is multi-class classification, which assigns multiple site categories (politicians, governmental organizations, television shows, and companies) to a page.

(3) AmazonComputer is a network of Amazon products. The nodes represent "Computer" products and an edge between two products encodes that the two products are co-purchased by the same customer. The node features are product reviews as bag-of-words. The task is to classify the product categories.

(4) OGBN_arxiv is a large-scale network of Computer Science (CS)

papers. The nodes represent a paper and each directed edge indicates a citation. Each paper comes with a 128-dimensional feature vector obtained by averaging the embeddings of words in its title and abstract. The task is to classify the subject areas.

GNN Classifiers. We employ three mainstream GNNs: (1) *Graph convolutional network* (GCN) [24], one of the classic message-passing GNNs; (2) *Graph attention networks* (GAT) [48] leverage attention mechanisms to dynamically weigh the importance of a node's neighbors during inference; and (3) *Graph isomorphism networks* (GIN) [52] with enhanced expressive power up to the Weisfeiler-Lehman (WL) graph isomorphism tests.

GNN Explainers. We have implemented the following.

(1) Our skyline exploratory query evaluation methods include two approximations ApxSX-OP (§ 4.2) and ApxSX-I (§ 4.2), and the diversification algorithm DivSX (§ **??**).

(2) GNNExplainer is a learning-based method that outputs masks for edges and node features by maximizing the mutual information between the probabilities predicted on the original and masked graph [54]. We induce explanatory subgraphs from the masks.

(3) PGExplainer learns edge masks to explain the GNNs. It trains a multilayer perception as the mask generator based on the learned features of the GNNs that require explanation. The loss function is defined in terms of mutual information [31].

(4) CF^2 is a model-agnostic GNN explanation framework that leverages both factual and counterfactual reasoning to learn feature and edge masks, producing effective and simple explanations [45].

(5) MOExp is a model-agnostic, bi-objective optimization framework that finds Pareto optimal explanations, striking a balance between "simulatability" (factual) and "counterfactual relevance" [29].

Evaluation Metrics. For all datasets and GNNs, we select three common explainability measures (Φ): fdl^+, fdl^-, and conc. As most GNN explainers are not designed to generate explanations for multiple explainability measures, we introduce three quality indicators (**QIs**) [27]. Consider a set of n GNN explainers, where each explainer A reports a set of explanatory subgraphs $\mathcal{G}_A$.

(1) **QI-1**: Integrated Preference Function (IPF) [6]. IPF score unifies and compares the quality of non-dominated set solutions with a weighted linear sum function. We define a normalized IPF of an explanation $\mathcal{G}_A$ from each GNN explainer A with a normalized single-objective score:

$$\mathsf{nIPF}(\mathcal{G}_A) = \frac{1}{|\mathcal{G}_A| \cdot |\Phi|} \sum_{G \in \mathcal{G}_A} \sum_{\phi \in \Phi} \phi(G) \tag{7}$$

(2) **QI-2**: Inverted Generational Distance (IGD) [14, 27], a most commonly used distance-based QI. It measures the distance from each solution to a reference set that contains top data points with theoretically achievable "ideal" values. We introduce IGD for explanations as follows. (a) We define a universal space $\mathcal{G} = \bigcup_{i \in [1,n]} \mathcal{G}_i$ from all the participating GNN explainers, and for each explainability measure $\phi \in \Phi$, induces a reference set $\mathcal{G}^k_\phi \in \mathcal{G}$ with explainatory subgraphs having the top-k values in ϕ. (b) The normalized IGD of an explanation $\mathcal{G}_A$ from an explainer A is defined as:

$$\mathsf{nIGD}(\mathcal{G}_A) = \frac{1}{k \cdot |\Phi|} \sum_{\phi \in \Phi} \sum_{G' \in \mathcal{G}^k_\phi} \min_{G \in \mathcal{G}_A} d(\phi(G), \phi(G'))$$

where $d(\cdot)$ is the Euclidean distance function.

(3) **QI-3**: Maximum Spread (MS) [27]. MS is a widely-adopted spread indicator that quantifies the range of the minimum and maximum values a solution can achieve in each objective. For a fair comparison, we introduce a normalized MS score using reference sets in QI-2. For each measure $\phi \in \Phi$, and an explanation $\mathcal{G}_A$, its normalized MS score on ϕ is computed as:

$$\mathsf{nMS}(\mathcal{G}_A)^\phi = \frac{\phi(G^{A*}_\phi)}{\phi(G^*_\phi)}$$

where G^*_ϕ is the explanatory subgraph with the best score on ϕ in the universal set $\mathcal{G}$, and G^{A*}_ϕ is the counterpart on ϕ from G_A.

(4) Efficiency. We report the total time cost of explanation generation. For learning-based approaches, this includes learning cost.

6.2 Experimental Results

Exp-1: Overall Explainability. We evaluate the overall performance of the GNN explainers using QIs.

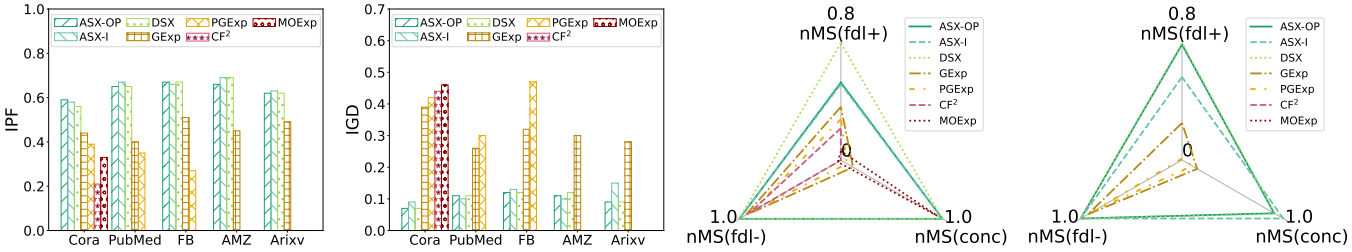
QI-1: IPF Scores. We report IPF scores (*bigger* is better) for all the GNN explainers in Fig. 3(a) for GCNs, with k=10.

(1) In general, ApxSX-OP outperforms a majority of baselines in aggregated explainability. For example, on Cora, ApxSX-OP outperforms GNNExplainer, PGExplainer, CF^2, and MOExp in IPF scores by 1.34, 1.51, 2.81, and 1.79 times, respectively. DivSX and ApxSX-I achieve comparable performance with ApxSX-OP.

(2) We also observe that the performance of GNNExplainer and PGExplainer are sensitive as the datasets vary. For example, on FacebookPage dataset, both baselines show a significant increase or decrease in IPF scores. In contrast, ApxSX-OP, ApxSX-I, and DivSX consistently achieve top IPF scores over all the datasets. This verifies that our methods are quite robust in generating representative high-quality explanations over different data sources.

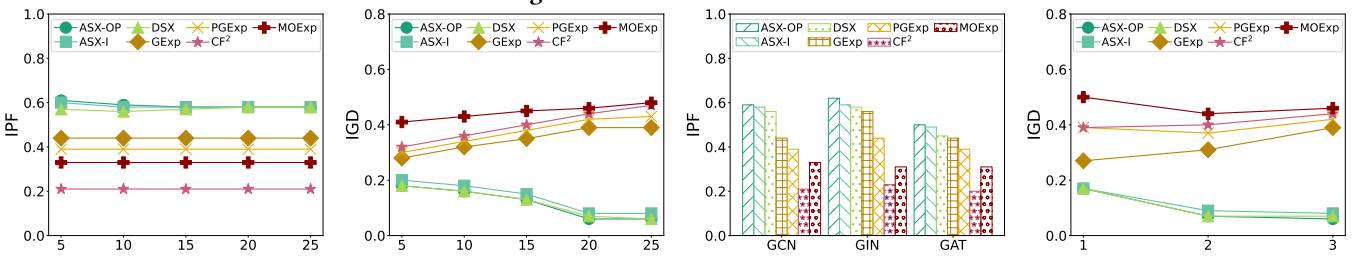
QI-2: IGD Scores. Fig. 3(b) reports the IGD scores (*smaller* is better) of the explanations for GCN-based classification. ApxSX-OP, ApxSX-I, and DivSX achieve the best in IGD scores among all GNN explainers, for all datasets. This verifies that our explanation method is able to consistently select top explanations from a space of high-quality explanations that are separately optimized over different measures. In particular, we found that ApxSX-OP, ApxSX-I, and DivSX are able to "recover" the top-k global optimal solutions for each individual explainability measures with high hit rate (not shown). For example, for conc, at least 9 out of 10 are correctly and consistently identified by ApxSX-OP over every dataset.

QI-3: nMX Scores. Figs. 3(c)& 3(d) visualize the normalized MS scores of the GNN explainers, over Cora and PubMed, respectively, where k = 10. (1) ApxSX-OP reports a large range and contributes to explanations that are either close or the exact best explanation, for each of the three measures. ApxSX-I and DivSX have comparable performance, yet with larger ranges due to its properly diversified solution. (2) DivSX, ApxSX-OP, and ApxSX-I contribute to the optimal fdl^+, fdl^-, and conc, respectively. On the other hand, each individual explainer performs worse for all measures, with a large gap. For example, in Cora, MOExp only achieves up to 3% of the best explanation (DivSX) over fdl^-.



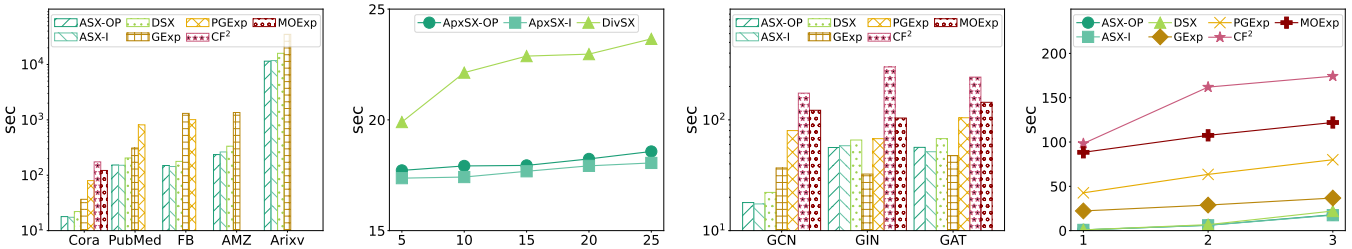
(a) IPF Overall Score (b) IGD Overall Score (c) Cora (d) PubMed

Figure 3: Overall Effectiveness.



(a) IPF varying k (b) IGD varying k (c) IPF varying GNNs (d) IGD varying L

Figure 4: Impact of factors.



(a) Efficiency (b) Scalability varying k (c) Scalability varying GNNs (d) Scalability varying L

Figure 5: Efficiency and Scalability.

Exp-2: Impact factors. We next report the impact of critical factors in skyline explanation generation over Cora, in terms of size of explanations k, GNN classes and the number of layers L.

Impact of k. Setting $\mathcal{M}$ as GCN-based classifier with 3 layers, we varied k from 5 to 25. We omit all baselines as they only generate a single explanation (hence remains insensitive to the change of k). (MOExp don't have size bound of k). Figs. 4(a) and 4(b) report the IPF and IGD scores of the GNN explainers. As k becomes larger, ApxSX-OP, ApxSX-I, and DivSX both achieve better scores and are able to stably generate explanations with higher quality. This is because larger k allows more dominating subgraphs to be verified, whenever possible for at least one measure. In contrast, MOExp is less sensitive to the growth of k, given that they may "stuck" at local optimal explanations that are optimized over a single measure, hence fail to improve the solution even with larger k.

Varying GNN classes. Fixing k=10, we report IPF scores of 3-layer GNN explainers for representative GNN classes including GCN, GAT and GIN over Cora. As verified in Fig 4(c), ApxSX-OP, ApxSX-I, and DivSX consistently outperform other explanations for different GNN classes. This verifies that our approaches remain stable in generating high-quality explanations for different types of GNNs. Our observations for IGD scores are consistent, hence omitted.

Varying number of layers. Fixing k=10, we report IGD scores of GCN explainers, with the layer L varies from 1 to 3 over Cora. Fig 4(d) shows that ApxSX-OP, ApxSX-I, and DivSX are able to achieve better IGD scores for generating explanations with more complex GNNs having "deeper" architecture. Indeed, as L becomes larger, all three are able to identify more and larger connected subgraphs that contribute to better fdl^+ and fdl^-, as well as the overall coverage of nodes and diversity, hence improving the overall explanation.

Exp-3: Efficiency. Using the same setting as in Fig. 3, we report the time cost of evaluating the skyline queries. Fig. 5(a) depicts the following.

(1) ApxSX-OP, ApxSX-I, and DivSX are, in general, feasible in generating high-quality explanations for GNN-based classification over large graphs. For example, it takes no more than 3.5 hours over all datasets. For OGBN_arxiv with 169,343 nodes and 1,166,243 edges, it takes up to 3.5 hours to generate explanations. This verifies the effectiveness of streaming-style subgraph generation, and the prioritization strategy.

(2) Comparing ApxSX-I and ApxSX-OP, we found that ApxSX-I achieves faster evaluation speed, the reason is that the datasets contain test nodes with "skewed" L-hop subgraph, i.e., less direct neighbors but have large multi-hop neighbors, which favors the

exploration of ApxSX-I. ApxSX-OP take a relatively longer time to converge to high-quality answers via onion peeling strategy, for nodes of interest with denser neighbors that are at "further" hops, while ApxSX-I take more time for nodes with high degrees, due to that it takes a longer time to verify smaller subgraphs.

(3) Comparing with DivSX, DivSX does not incur significant overhead despite that it pursues more diversified explanations. Indeed, the benefit of edge prioritization carries over to diversification process, and the incremental maintenance of the explanations reduces unnecessary verification.

(4) ApxSX-OP, ApxSX-I, and DivSX outperforms all learning-based GNN explanations. For example, ApxSX-OP (resp. DivSX) on average outperforms GNNExplainer, PGExplainer, CF^2, and MOExp by 2.05, 4.46, 9.73, 6.81 (resp. 1.62, 3.61, 7.88, and 5.51) times, respectively. Moreover, ApxSX-OP has a larger performance gain over GNNExplainer and PGExplainer over larger graphs. CF^2 and MOExp fail to generate explanations with memory issues or infeasible time cost. Indeed, The learning-based methods require more learning overhead as the graphs become larger and denser.

Exp-4: Scalability. We next evaluate the impact of k, GNN classes and L to the efficiency.

Varying k. For learning-based methods, they are in general not sensitive to the change of k, hence we omit their performances. Using the same setting as in Figs. 4(a)& 4(b), we report the time cost of GNN explainers, ApxSX-OP, ApxSX-I and DivSX in Fig. 5(b). As expected, for all three methods, it takes a longer time to maintain a skyline explanation with larger size k, each time a new subgraph passes the verification. However, the difference is insignificant. DivSX is more sensitive to k due to the additional computation of distance. On the other hand, all take up to 23 seconds to maintain the explanations with k varied to 25.

Varying L and GNN classes. Following the same setting of their counterparts in Figs. 4(c) and 4(d), Figs. 5(c) and 5(d) reports the time cost of ApxSX-OP, ApxSX-I and DivSX. (1) All the methods take a longer time to generate explanations as L becomes larger, due to that more subgraphs need to be verified from larger induced L-hop neighbors, and the increased overhead from verification that invokes GNN inference, over larger subgraphs. (2) The three methods take least time. This is consistent with our observation that the verification of GCN is the most efficient among all GNN classes, indicating an overall small verification cost.

Exp-5: Case Analysis. We next showcase of the application of our explanation methods.

Diversified Interpretation. A user is interested in finding "Why" a product v_4 is labeled "PC Gaming" by a GCN. GNN explainers that optimize a single measure *e.g.*,GNNExplainerreturn explanations (as shown in G_6) that simply reveals a fact that v_4 is co-purchased with "Components", a "one-sided" interpretation. In contrast, DivSX identifies an explanation that reveals more comprehensive interpretation with two explanatory subgraphs (both factual) $\{G_7, G_8\}$, which reveals two co-purchasing patterns bridging v_4 with not only "Components" for "Monitors" (by G_7), but also to "Accessories" of "Laptops". Indeed, a closer inspection confirms that the former indicates gamers who prefer DIY their gaming PC with high-refresh

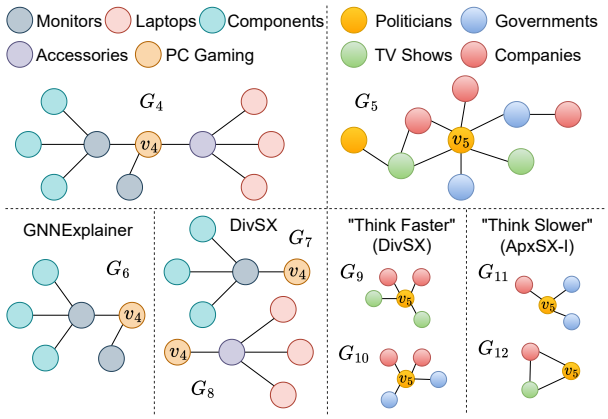


Figure 6: Case Analysis. Left: Diversified skyline explanation captures the distinct subgraphs that contribute to the classification of test node v_t. Right: Different edge explore strategies show different preferences for returned skyline explanation.

rate monitors designed for gaming; and the latter indicates v_4 is a gaming laptop that needs frequent maintenance with laptop accessories. This verifies DivSX is able to provide more comprehensive interpretation for GNNs.

Think slower, think faster? Our second case compare the application scenarios of ApxSX-OP and ApxSX-I. We observe that ApxSX-OP, with onion peeling, cater users' need who "think fast" and is good at quick generation of larger, factual explanations as convincing evidences at first glance [29]. ApxSX-I, on the other hand tends to produce at its early stage smaller, counterfactual explanations due to its containment of edges closer to targeted ones via edge insertions, which are more likely to affect GNN behaviors if removed. This caters users who "think slower" and seeks in-depth explanations that are counterfactual [29]. For example, in answering "why" node v_5 is recognized as a "Politician" site, G_9 and G_{10} are among skyline explanations from DivSX at early stage, revealing the fact that v_5 frequently interact with TV shows, Governments and Companies, covering its ego-networks as quick facts. G_{11} and G_{12}, which are reported by ApxSX-I in its early stage, are relatively smaller counterfactual explanations that suggest interaction patterns with more complex structures (*e.g.,* triangles).

7 CONCLUSION

We have proposed a class of skyline explanatory query that allows users to generate explanatory subgraphs that simutaneously optimize multiple preference explainability measures, for GNN-based classification. We have shown that the problem of skyline explanation generation, which evaluates such queries, remains nontrivial even for polynomially bounded explanation space and a measure space of 3 measures. Despite the hardness, we have introduced feasible explanation generation algorithms, with provable quality guarantees on explanations, for both skyline query evaluation and its diversified counterpart. Our experimental study has verified that our methods are feasible and robust in generating high-quality explanations over large-scale GNN-based classification. Our case study also verifies that they can interpret the result of GNN-based

classification with more comprehensive explanations, compared with single-objective GNN explaners. A future topic is to investigate distributed explanation generation algorithms for skyline explanatory query.

REFERENCES

[1] 2024. Code and datasets. *https://anonymous.4open.science/r/se1.*

[2] G. Ausiello, N. Boria, A. Giannakos, G. Lucarelli, and V.Th. Paschos. 2012. Online maximum k-coverage. *Discrete Applied Mathematics* 160, 13 (2012), 1901–1913.

[3] Ashwinkumar Badanidiyuru, Baharan Mirzasoleiman, Amin Karbasi, and Andreas Krause. 2014. Streaming submodular maximization: massive data summarization on the fly. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, New York, USA). Association for Computing Machinery, New York, NY, USA, 671–680.

[4] Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. 2024. The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset. In *KDD Workshop on Machine Learning in Finance* (Barcelona, Spain). Association for Computing Machinery, New York, NY, USA.

[5] S. Borzsony, D. Kossmann, and K. Stocker. 2001. The Skyline operator. In *Proceedings 17th International Conference on Data Engineering*. 421–430.

[6] W Matthew Carlyle, John W Fowler, Esma S Gel, and Bosun Kim. 2003. Quantitative comparison of approximate solution sets for bi-criteria optimization problems. *Decision Sciences* 34, 1 (2003), 63–82.

[7] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. 2020. Scalable graph neural networks via bidirectional propagation. *NeurIPS* (2020).

[8] Tingyang Chen, Dazhuo Qiu, Yinghui Wu, Arijit Khan, Xiangyu Ke, and Yunjun Gao. [n.d.]. View-based explanations for graph neural networks. *Proceedings of the ACM on Management of Data* ([n. d.]).

[9] Ling Cheng, Feida Zhu, Yong Wang, Ruicheng Liang, and Huiwen Liu. 2023. Evolve Path Tracer: Early Detection of Malicious Addresses in Cryptocurrency. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA). Association for Computing Machinery, New York, NY, USA, 3889–3900.

[10] Kenneth Chircop and David Zammit-Mangion. 2013. On e-constraint based methods for the generation of Pareto frontiers. (2013).

[11] Eunjoon Cho, Seth A. Myers, and Jure Leskovec. 2011. Friendship and mobility: user movement in location-based social networks. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Diego, California, USA). Association for Computing Machinery, New York, NY, USA, 1082–1090.

[12] Jan Chomicki, Paolo Ciaccia, and Niccolo' Meneghetti. 2013. Skyline queries, front and back. *SIGMOD Rec.* 42, 3 (oct 2013), 6–18.

[13] Paolo Ciaccia and Davide Martinenghi. 2017. Reconciling skyline and ranking queries. *Proc. VLDB Endow.* 10, 11 (aug 2017), 1454–1465.

[14] Carlos A Coello Coello and Margarita Reyes Sierra. 2004. A study of the parallelization of a coevolutionary multi-objective evolutionary algorithm. In *MICAI 2004: Advances in Artificial Intelligence: Third Mexican International Conference on Artificial Intelligence, Mexico City, Mexico, April 26-30, 2004. Proceedings 3*. 688–697.

[15] Youssef Elmougy and Ling Liu. 2023. Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) *(KDD '23)*. Association for Computing Machinery, New York, NY, USA, 3979–3990.

[16] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *The World Wide Web Conference* (San Francisco, CA, USA). Association for Computing Machinery, New York, NY, USA, 417–426.

[17] Santo Fortunato. 2010. Community Detection in Graphs. *Physics reports* 486, 3-5 (2010), 75–174.

[18] Floris Geerts. 2023. A Query Language Perspective on Graph Learning. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 373–379.

[19] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.

[20] H. He, Y. Ji, and H. Huang. 2022. Illuminati: Towards Explaining Graph Neural Networks for Cybersecurity Analysis. In *2022 IEEE 7th European Symposium on Security and Privacy*. IEEE Computer Society, Los Alamitos, CA, USA, 74–89. https://doi.ieeecomputersociety.org/10.1109/EuroSP53844.2022.00013

[21] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.

[22] C-L Hwang and Abu Syed Md Masud. 2012. *Multiple objective decision making—methods and applications: a state-of-the-art survey*. Vol. 164. Springer Science & Business Media.

[23] Shant Karakashian, Berthe Y Choueiry, and Stephen G Hartke. 2013. An algorithm for generating all connected subgraphs with k vertices of a graph. *Lincoln, NE* 10, 2505515.2505560 (2013).

[24] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[25] Harold William Kuhn and Albert William Tucker. 1953. *Contributions to the Theory of Games*. Number 28. Princeton University Press.

[26] Marco Laumanns, Lothar Thiele, Eckart Zitzler, and Kalyanmoy Deb. 2002. Archiving with guaranteed convergence and diversity in multi-objective optimization. In *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation* (New York City, New York). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 439–447.

[27] Miqing Li and Xin Yao. 2019. Quality evaluation of solution sets in multiobjective optimisation: A survey. *ACM Computing Surveys (CSUR)* 52, 2 (2019), 1–38.

[28] Xuemin Lin, Yidong Yuan, Qing Zhang, and Ying Zhang. 2006. Selecting stars: The k most representative skyline operator. In *ICDE*. 86–95.

[29] Yifei Liu, Chao Chen, Yazheng Liu, Xi Zhang, and Sihong Xie. 2021. Multi-objective explanations of GNN predictions. In *2021 IEEE International Conference on Data Mining (ICDM)*. 409–418.

[30] Ana Lucic, Maartje A Ter Hoeve, Gabriele Tolomei, Maarten De Rijke, and Fabrizio Silvestri. 2022. Cf-gnnexplainer: Counterfactual explanations for graph neural networks. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4499–4511.

[31] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. *Advances in neural information processing systems* 33 (2020), 19620–19631.

[32] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the Construction of Internet Portals with Machine Learning. *Information Retrieval* 3 (2000), 127–163.

[33] Ramaravind K Mothilal, Amit Sharma, and Chenhao Tan. 2020. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proceedings of the 2020 conference on fairness, accountability, and transparency*. 607–617.

[34] Shiyu Ouyang, Qianlan Bai, Hui Feng, and Bo Hu. 2024. Bitcoin Money Laundering Detection via Subgraph Contrastive Learning. *Entropy* 26, 3 (2024).

[35] C.H. Papadimitriou and M. Yannakakis. 2000. On the approximability of trade-offs and optimal access of Web sources. In *Proceedings 41st Annual Symposium on Foundations of Computer Science*. 86–92.

[36] P. Jonathon Phillips, Carina Hahn, Peter Fontana, Amy Yates, Kristen K. Greene, David Broniatowski, and Mark A. Przybocki. 2021. Four Principles of Explainable Artificial Intelligence.

[37] Nadia Pocher, Mirko Zichichi, Fabio Merizzi, Muhammad Zohaib Shafiq, and Stefano Ferretti. 2023. Detecting anomalous cryptocurrency transactions: An AML/CFT application of machine learning-based forensics. *Electronic Markets* 33, 1 (2023), 37.

[38] Sudeep Pushpakom, Francesco Iorio, Patrick A Eyers, K Jane Escott, Shirley Hopper, Andrew Wells, Andrew Doig, Tim Guilliams, Joanna Latimer, Christine McNamee, et al. 2019. Drug repurposing: progress, challenges and recommendations. *Nature reviews Drug discovery* 18, 1 (2019), 41–58.

[39] Dazhuo Qiu, Mengying Wang, Arijit Khan, and Yinghui Wu. 2024. Generating Robust Counterfactual Witnesses for Graph Neural Networks. In *ICDE*. 3351–3363.

[40] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-Scale Attributed Node Embedding. *Journal of Complex Networks* 9, 2 (2021), cnab014.

[41] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. 2020. Interpreting graph neural networks for NLP with differentiable edge masking. *arXiv preprint arXiv:2010.00577* (2020).

[42] Oliver Schütze and Carlos Hernández. 2021. Computing ϵ-(approximate) Pareto Fronts. In *Archiving Strategies for Evolutionary Multi-objective Optimization Algorithms*. 41–66.

[43] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–106.

[44] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of Graph Neural Network Evaluation. *Relational Representation Learning Workshop, NeurIPS 2018* (2018).

[45] Juntao Tan, Shijie Geng, Zuohui Fu, Yingqiang Ge, Shuyuan Xu, Yunqi Li, and Yongfeng Zhang. 2022. Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning. In *Proceedings of the ACM web conference 2022*. 1018–1027.

[46] George Tsaggouris and Christos Zaroliagis. 2009. Multiobjective optimization: Improved FPTAS for shortest paths and non-linear objectives with applications. *Theory of Computing Systems* 45, 1 (2009), 162–186.

[47] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.

[48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

[49] Yuyang Wang, Zijie Li, and Amir Barati Farimani. 2023. *Graph Neural Networks for Molecules*. Springer International Publishing, Cham, 21–66.

[50] Lanning Wei, Huan Zhao, Zhiqiang He, and Quanming Yao. 2023. Neural Architecture Search for GNN-Based Graph Classification. *ACM Trans. Inf. Syst.* 42, 1, Article 1 (aug 2023), 29 pages.

[51] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).

[52] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.

[53] Zhengyi Yang, Longbin Lai, Xuemin Lin, Kongzhang Hao, and Wenjie Zhang. 2021. HUGE: An Efficient and Scalable Subgraph Enumeration System. In *SIGMOD*.

[54] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GNNExplainer: Generating Explanations for Graph Neural Networks. *Advances in neural information processing systems* 32 (2019).

[55] Jiaxuan You, Bowen Liu, Rex Ying, Vijay Pande, and Jure Leskovec. 2018. Graph convolutional policy network for goal-directed molecular graph generation. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada). Curran Associates Inc., Red Hook, NY, USA, 6412–6422.

[56] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2023. Explainability in graph neural networks: A taxonomic survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 5 (2023), 5782–5799.

[57] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *International conference on machine learning*. PMLR.

[58] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in neural information processing systems* 31 (2018).

[59] Shichang Zhang, Yozen Liu, Neil Shah, and Yizhou Sun. 2022. Gstarx: Explaining graph neural networks with structure-aware cooperative games. *Advances in Neural Information Processing Systems* 35 (2022).

[60] Youmin Zhang, Qun Liu, Guoyin Wang, William K Cheung, and Li Liu. 2024. GEAR: Learning graph neural network explainer via adjusting gradients. *Knowledge-Based Systems* 302 (2024), 112368.

APPENDIX: PROOFS AND ALGORITHMS